

Nama: Alchalisa Fathinna
NIM: 245150407111066
MK: Pengembangan Aplikasi Mobile - F

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawaban: Pada project XML/View, objek yang diubah secara langsung adalah properti text dari TextView. Baris kodenya adalah: `tvMessage.text = "Tombol sudah diklik"`

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawaban: Pada project Compose, nilai yang diubah adalah *state* message. Baris kodenya adalah: `message = "Tombol sudah diklik"`.

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Jawaban: Jetpack Compose menggunakan pendekatan deklaratif, di mana pengembang tidak mengubah objek UI secara langsung. Sebaliknya, pengembang mendeklarasikan tampilan berdasarkan suatu *state*, dan ketika *state* itu berubah, Compose akan otomatis memperbarui bagian UI yang terpengaruh (*recomposition*).

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawaban: Polanya adalah: Event terjadi -> Program mencari View (misal via `findViewById`) -> Program memerintahkan objek tersebut untuk mengubah propertinya (perubahan tampilan).

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawaban: Polanya adalah: Event terjadi -> Program mengubah *state* yang menjadi sumber data UI -> Compose mendeteksi perubahan dan menjalankan *recomposition* -> Tampilan diperbarui.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jawaban: Jika menggunakan `var message = "Belum diklik"`, maka saat tombol diklik, tampilan teks tidak akan berubah. Hal ini karena meskipun nilai variabelnya berubah, variabel lokal biasa tidak otomatis meminta Compose untuk menggambar ulang UI (tidak memicu *recomposition*).

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Jawaban:

- Imperatif: Pengembang harus secara aktif mencari komponen UI dan memerintahkan sistem *bagaimana* cara mengubah propertinya langkah demi langkah.
- Deklaratif: Pengembang hanya mendeskripsikan *apa* yang harus ditampilkan oleh UI berdasarkan kondisi data (*state*) saat ini, dan membiarkan sistem yang mengurus proses pembaruannya secara otomatis.

Event Handling State

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.
2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawaban: Setiap kali diklik, LogCat akan menampilkan tulisan: "Counter berubah 1", "Counter berubah 2", dan seterusnya.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawaban: Teks pada layar akan tetap bertuliskan "Counter: 0" dan tidak berubah.

4. Apa yang menyebabkan hal tersebut terjadi?

Jawaban: Variabel count hanyalah variabel biasa, sehingga perubahannya tidak diamati oleh sistem Compose untuk memicu penggambaran ulang UI.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.
6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

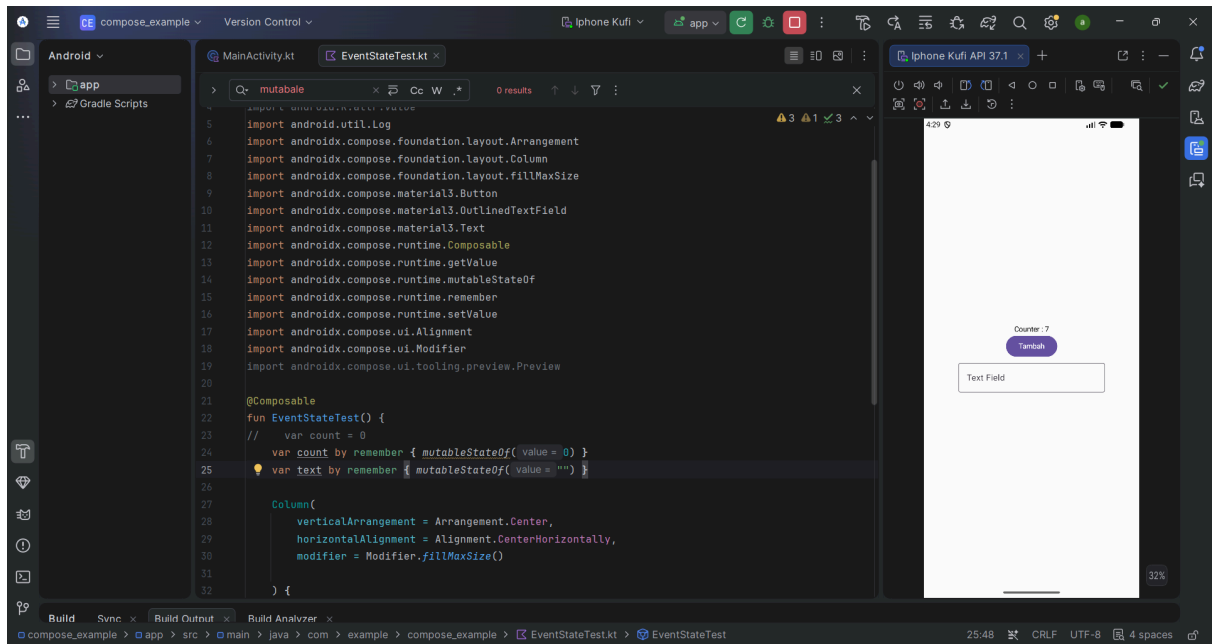
`var count = 0`

menjadi:

`var count by remember { mutableStateOf(0) }`

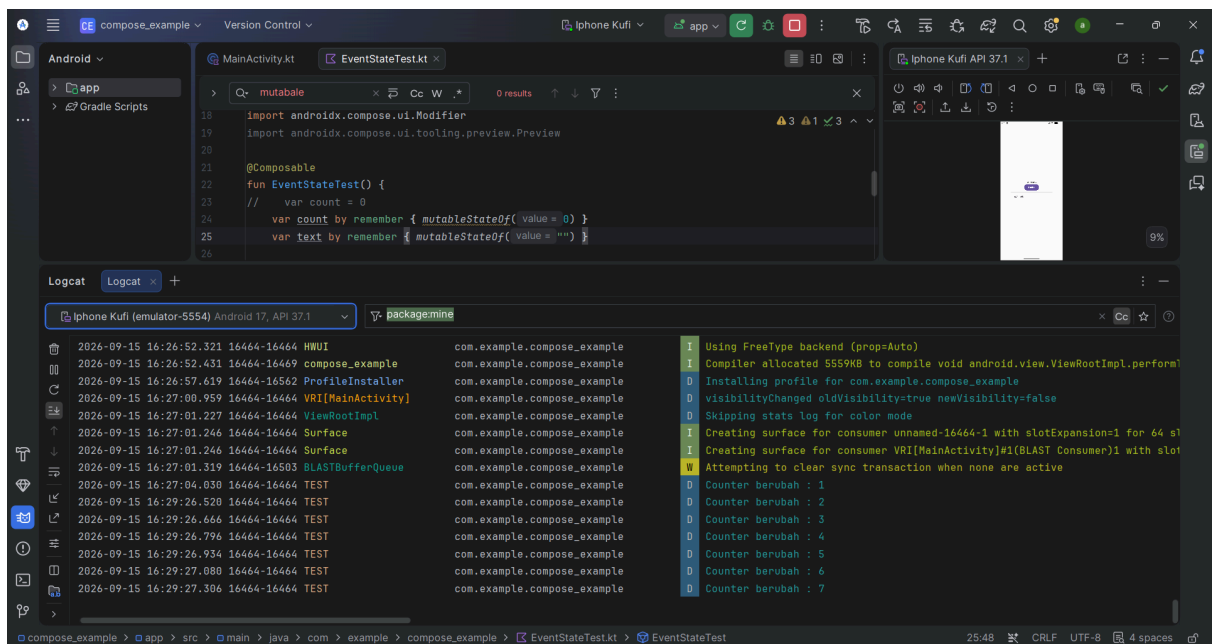
7. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawaban:



8. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawaban:



9. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawaban: Angka pada layar sekarang berhasil berubah menjadi "Counter: 1", "Counter: 2", dst.

10. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

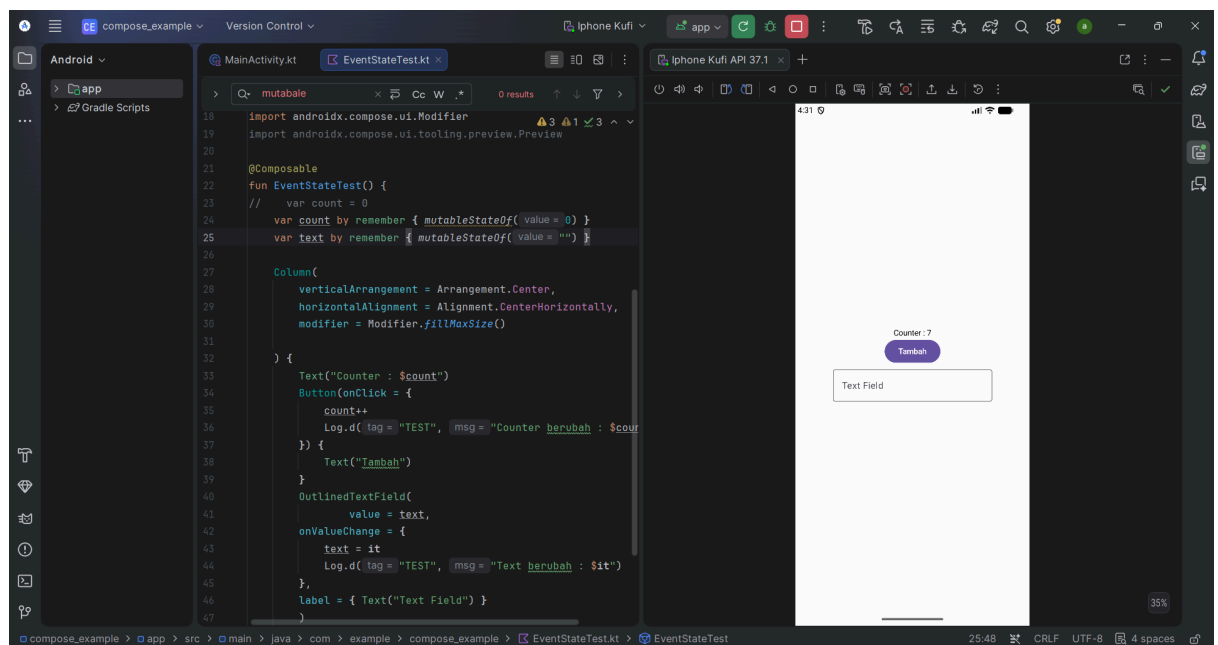
Jawaban: Variabel count sekarang menggunakan mutableStateOf() yang merupakan *state* yang dapat diamati. Ketika nilainya diubah, Compose mengetahui ada perubahan data dan otomatis memicu *recomposition*.

11. Apa yang dimaksud recomposition pada Compose?

Jawaban: Recomposition adalah proses di mana Jetpack Compose memperbarui atau menggambar ulang bagian UI yang bergantung pada suatu *state* ketika *state* tersebut nilainya berubah.

12. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:
13. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawaban:



14. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawaban: Setiap huruf diketik, LogCat akan mencetak perubahan *real-time*, misalnya: "Text berubah: h", "Text berubah: ha", "Text berubah: ha1", dst.

15. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawaban: Karakter yang diinputkan pengguna akan langsung muncul dan memperbarui teks di dalam kolom Text Field.

16. Apa yang menyebabkan hal tersebut terjadi?

Jawaban: Mengetik memicu event `onValueChange` yang secara langsung mengubah *state* text. Perubahan state ini memicu recomposition sehingga kolom menampilkan input terbaru.

17. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Jawaban: count (bertipe Int) dan text (bertipe String).

18. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Jawaban: `onClick` (pada `Button`) dan `onValueChange` (pada `OutlinedTextField`).

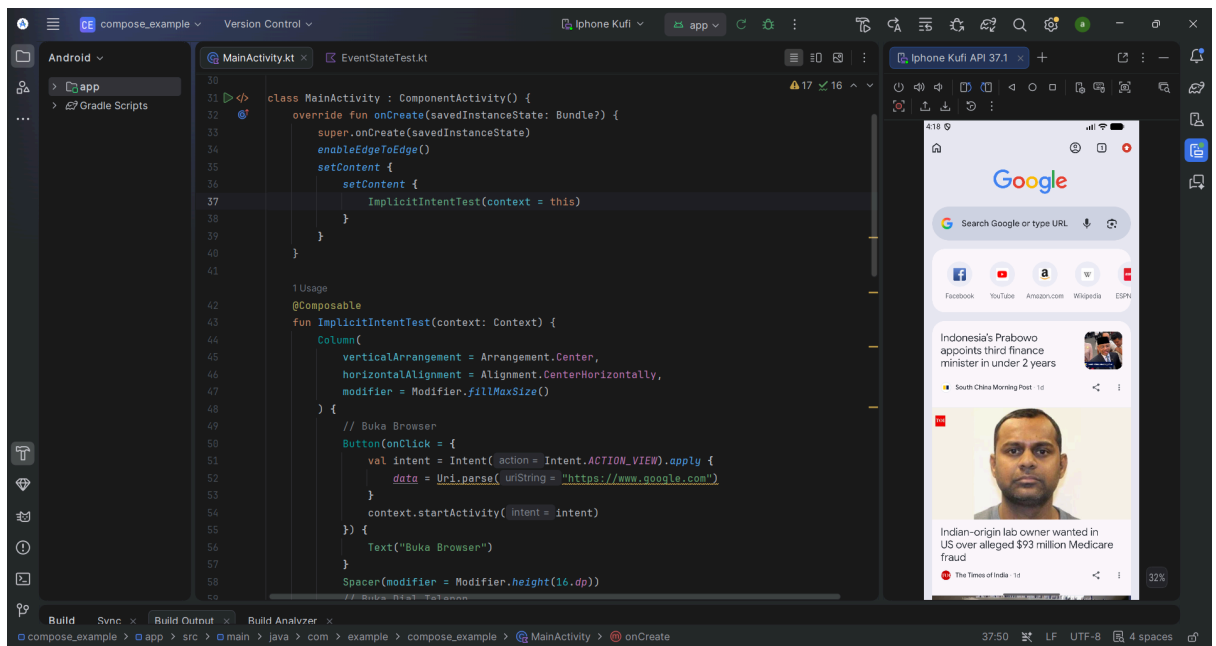
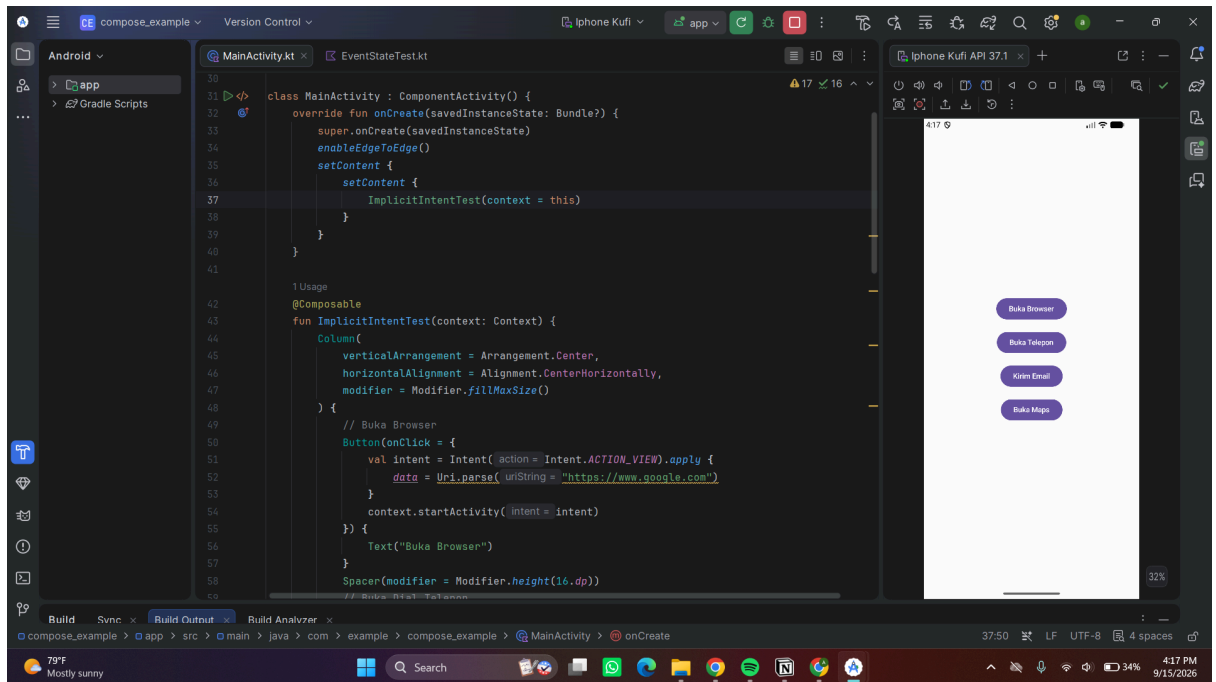
19. Kapan event - event tersebut dijalankan?

Jawaban: Event `onClick` dijalankan ketika pengguna menekan tombol, sedangkan event `onValueChange` dijalankan setiap kali ada perubahan input (mengetik atau menghapus) pada kolom teks.

Implisit Intent

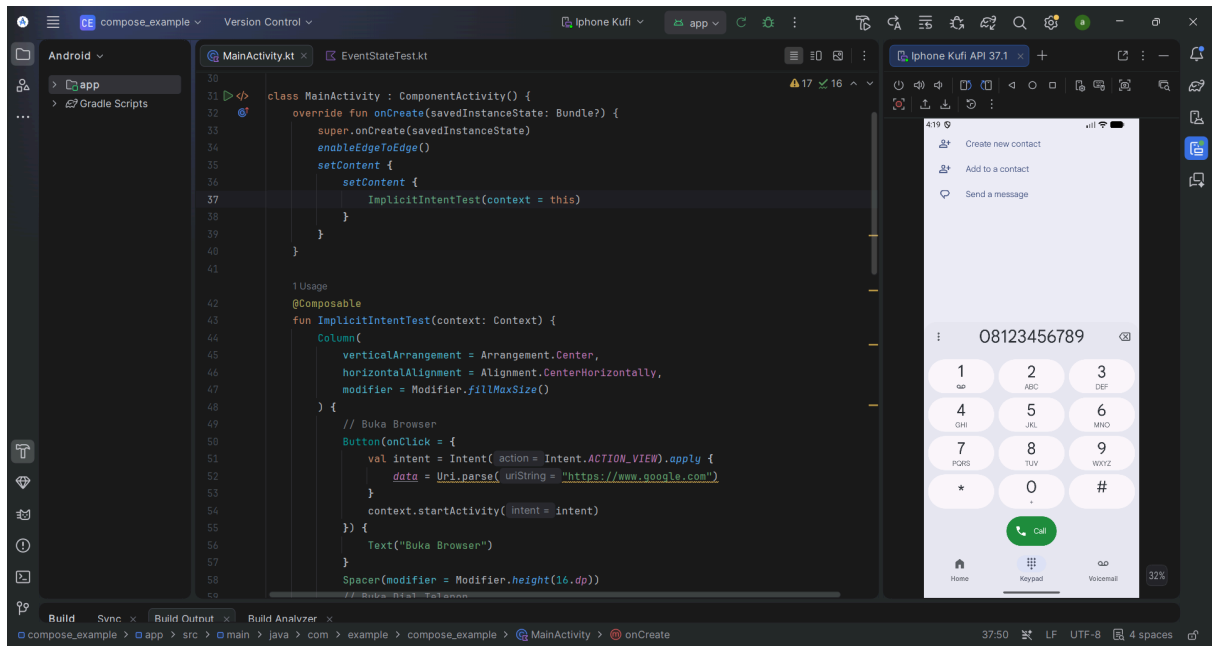
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:
2. Klik Buka Browser

Jawaban:



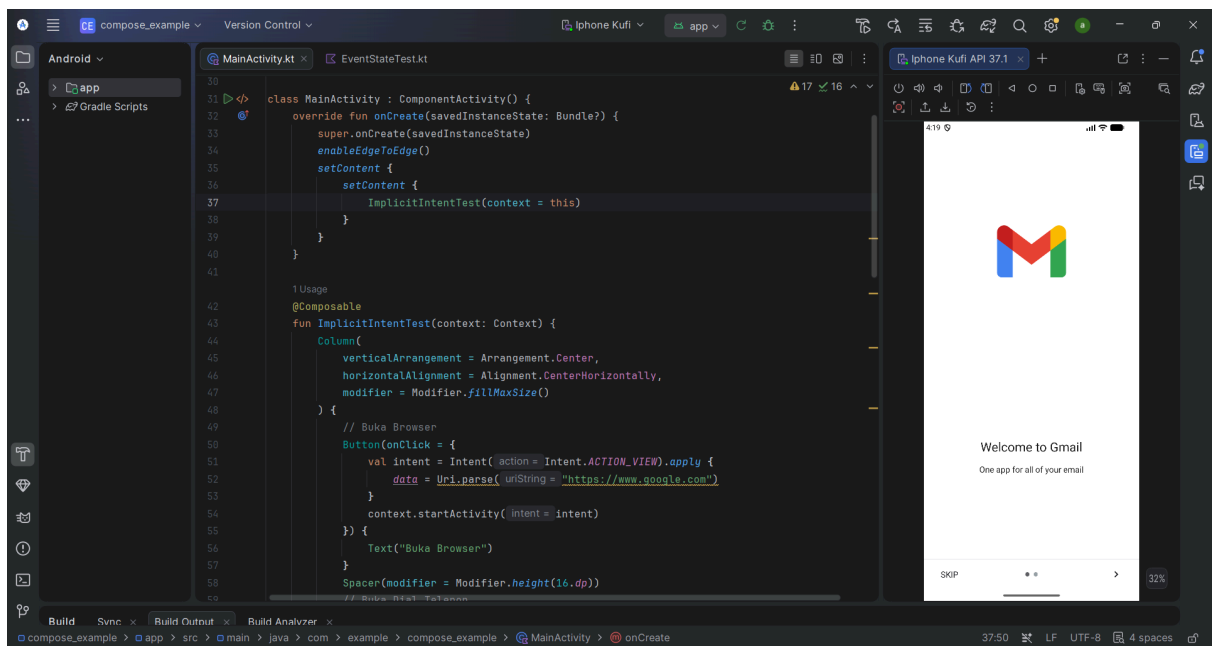
3. Klik Buka Telepon

Jawaban:



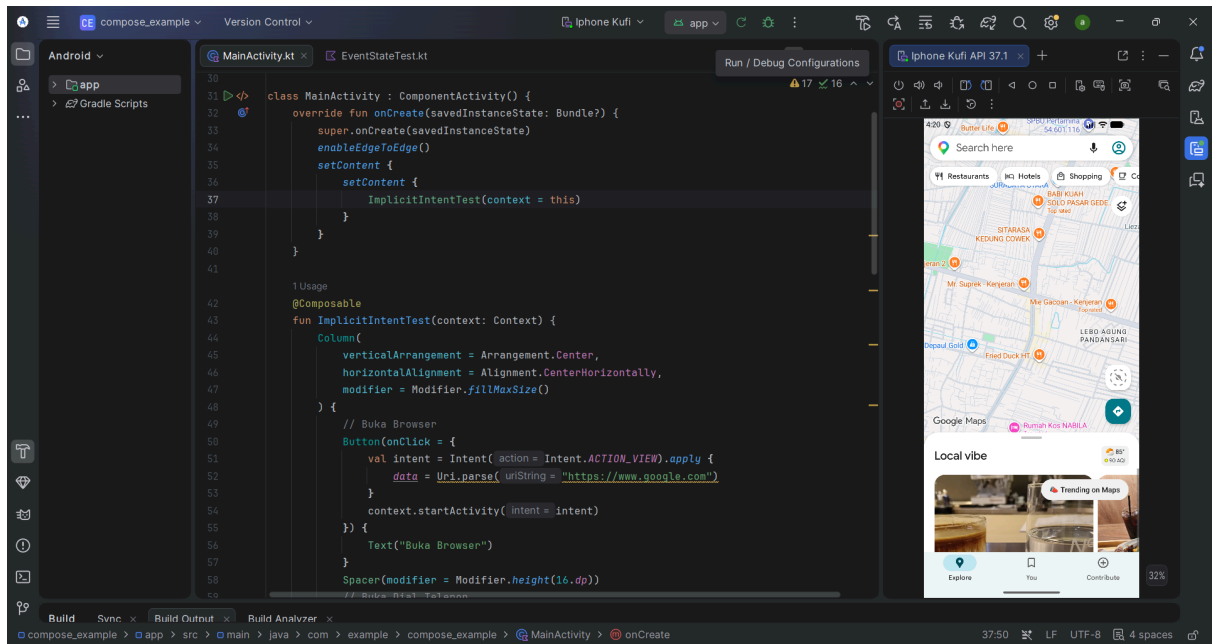
4. Klik Kirim Email

Jawaban:



5. Klik Buka Maps

Jawaban:



6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya.
7. Pada kode yang digunakan, terdapat beberapa action seperti:
 - Intent.ACTION_VIEW
 - Intent.ACTION_DIAL
 - Intent.ACTION_SENDTO

Apa perbedaan fungsi dari masing-masing action intent tersebut?

Jawaban:

- Intent.ACTION_VIEW: Digunakan untuk menampilkan atau melihat suatu data, seperti membuka alamat web di browser atau koordinat di peta.
- Intent.ACTION_DIAL: Digunakan khusus untuk membuka aplikasi telepon (dialer) dengan nomor yang sudah terisi, namun belum memulai panggilan.
- Intent.ACTION_SENDTO: Digunakan untuk mengirim data/pesan ke penerima spesifik, seperti membuka aplikasi email

8. Pada kode yang digunakan, terdapat beberapa data seperti:
 - data = Uri.parse("mailto:test@gmail.com")
 - data = Uri.parse("https://www.google.com")
 - data = Uri.parse("geo:-7.250445,112.768845")
9. Apa fungsi data pada intent ?

Jawaban: Data (berupa format Uri) berfungsi untuk memberikan informasi spesifik mengenai apa yang harus diproses oleh aksi tersebut, seperti alamat web yang dituju, nomor telepon, atau alamat email.

10. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Jawaban: Action menentukan jenis tindakan yang ingin dilakukan (misal: melihat atau mengirim), sedangkan Data menentukan target dari tindakan tersebut. Sistem

Android menggunakan kombinasi keduanya untuk memunculkan daftar aplikasi yang relevan dan mampu menangani permintaan tersebut.